DESENVOLVIMENTO DE API BACK-END · ENGENHARIA DE SOFTWARE · 6º PERÍODO

Semana 09
Autenticação JWT e autorização por papel

Apostila de estudo: teoria, exemplos, construção com IA e exercícios

Na semana passada a senha passou a ser guardada como hash BCrypt e o cadastro ganhou validação. Falta responder a três perguntas que toda API protegida precisa resolver a cada requisição: quem está chamando, o que essa pessoa pode fazer e de quem é o recurso que ela quer mexer. Esta apostila explica o token JWT por dentro, mostra como o Spring Security valida esse token sem que você escreva um filtro, e separa com cuidado a autorização por papel da autorização por dono, que é onde a maior parte das APIs reais falha.

Conteúdo

1. Objetivos da semana
2. Autenticar e autorizar
3. Sessão com estado e token sem estado
4. O JWT por dentro
5. HS256 e RS256
6. Claims registradas e privadas
7. Expiração, revogação e refresh token
8. OAuth 2.0 e resource server
9. Papel, dono e menor privilégio
10. 401 ou 403
11. Aplicação no ServiceHub
12. Construindo com IA
13. Glossário
14. Exercícios
15. Para estudar

1. Objetivos da semana

↑ conteúdo

Ao final desta semana você deve ser capaz de:

1. comparar sessão com estado e token sem estado e explicar por que uma API REST prefere o token;
2. ler um JWT, dizer o que a assinatura garante e o que ela não garante, e escolher entre HS256 e RS256;
3. configurar emissão e validação de JWT com o suporte nativo do Spring Security, sem biblioteca extra e sem filtro escrito à mão;
4. aplicar autorização por papel com `@PreAuthorize` e autorização por dono com checagem explícita no service, evitando BOLA;
5. decidir entre 401 e 403 e devolver os dois no formato ProblemDetail do contrato.

2. Autenticar e autorizar

↑ conteúdo

Duas palavras parecidas descrevem etapas diferentes. **Autenticação** responde "quem é você?": o sistema confere uma prova de identidade, como senha, token ou certificado. **Autorização** responde "o que você pode fazer?": com a identidade já conhecida, o sistema decide se aquela operação, sobre aquele objeto, é permitida.

A ordem importa. Não faz sentido perguntar se alguém pode apagar um serviço antes de saber quem é esse alguém. Por isso as falhas também são diferentes: falhar na autenticação gera um erro que convida o cliente a se identificar; falhar na autorização gera um erro que diz que, mesmo identificado, ele não tem permissão. A seção 10 transforma essa diferença em status HTTP.

> **ANALOGIA**
>
> Na portaria de uma empresa, o crachá com foto é a autenticação: prova que você é você. A catraca que libera só certos andares é a autorização. Ter crachá não abre todas as portas; e uma porta trancada não questiona quem você é, apenas recusa a entrada.

Uma API costuma autorizar em duas camadas. A primeira olha o **papel** do usuário (cliente, prestador, administrador). A segunda olha a **relação** entre o usuário e o objeto (é o dono? participa desta ordem?). A primeira é fácil de lembrar; a segunda é a que tutoriais e código gerado por IA mais esquecem.

3. Sessão com estado e token sem estado

↑ conteúdo

O HTTP não lembra de nada entre uma requisição e outra. Depois do login, alguma coisa precisa fazer a ponte e dizer ao servidor, a cada chamada, que é a mesma pessoa. Há duas famílias de solução, e a diferença entre elas está em *quem guarda o estado*. "Estado", aqui, é a memória que o servidor tem sobre o cliente entre uma requisição e outra.



As duas formas de manter o usuário identificado entre requisições.

Na **sessão**, o servidor cria um registro na memória (ou num banco de sessões) e entrega ao cliente só um identificador aleatório, em geral num cookie. A cada chamada o servidor procura esse id na sua tabela. No **token**, o servidor monta os dados do usuário, assina e entrega tudo ao cliente. A cada chamada o servidor apenas confere a assinatura e a validade; não precisa lembrar de nada.

Critério	Sessão no servidor	Token (JWT)
Onde fica o estado	Na memória ou banco do servidor	No próprio token, com o cliente
Várias instâncias da API	Exigem sessão compartilhada ou balanceador "grudento"	Qualquer instância valida, basta ter a chave
Como viaja	Cookie de sessão, enviado pelo navegador sozinho	Header `Authorization: Bearer <token>`
Revogar	Apagar a sessão: efeito imediato	Esperar o vencimento ou consultar algo no servidor
CSRF	Precisa de proteção, porque o navegador manda o cookie sozinho	Não se aplica ao header Bearer, que o código cliente envia de propósito

O token combina com a restrição *stateless* do REST, vista na Semana 01: cada requisição carrega tudo o que o servidor precisa. O preço aparece na linha "revogar", e a seção 7 volta a ela.

Uma imagem ajuda: a sessão é a comanda que fica com o garçom; o token é a pulseira do parque de diversões, que você mostra em cada brinquedo. **No ServiceHub**, a configuração de segurança declara `SessionCreationPolicy.STATELESS`: o Spring não cria sessão HTTP, e cada requisição protegida precisa trazer o token.

4. O JWT por dentro

↑ conteúdo

JWT (*JSON Web Token*, definido na RFC 7519) é um formato compacto para transportar afirmações sobre alguém entre duas partes. O JWT que usamos é assinado segundo o padrão **JWS** (*JSON Web Signature*, RFC 7515). Na prática ele é um texto com três partes separadas por ponto:

```
eyJhbGciOiJIUzI1NiJ9 . eyJzdWIiOiJhbmEifQ . <assinatura>
       header                payload            assinatura
```

Parte	O que contém	Exemplo decodificado
Header	JSON que diz o algoritmo da assinatura (`alg`) e, opcionalmente, o tipo (`typ`).	`{"alg":"HS256"}`
Payload	JSON com as afirmações sobre o usuário, chamadas **claims**.	`{"sub":"ana"}`
Assinatura	Calculada sobre o texto `header.payload` com uma chave. Mudou um caractere, a assinatura deixa de conferir.	bytes codificados, sem significado legível

Base64URL: codificação, não criptografia

Cada parte é codificada em **Base64URL**, uma variante do Base64 adaptada para URLs: troca `+` por `-`, `/` por `_` e remove o `=` de preenchimento do fim. O objetivo é que o token viaje em headers e URLs sem precisar de escape. Qualquer pessoa reverte essa codificação: no console do navegador, `atob("eyJhbGciOiJIUzI1NiJ9")` devolve `{"alg":"HS256"}`.

CONFUSÃO COMUM

"O JWT é criptografado." Não é. O payload de um JWT assinado é legível por qualquer um que tenha o token. Existe um formato criptografado, o JWE, mas é outro padrão e não é o que usamos. Retome a tabela da Semana 08: Base64URL é *codificação* (reversível sem chave), a assinatura usa *hash* com chave, e nada ali é *criptografia* do conteúdo.

O que a assinatura garante e o que não garante

Garante

- **Integridade:** ninguém alterou header nem payload depois da emissão.
- **Origem:** foi emitido por quem tem a chave.
- **Validade**, junto com o claim `exp`: ainda está dentro do prazo.

Não garante

- **Sigilo:** o payload não é cifrado.
- **Posse legítima:** quem rouba o token o usa como se fosse o dono.
- **Dado atual:** o papel ou o status do usuário podem ter mudado depois da emissão.

Dessas limitações saem quatro regras práticas: o token só trafega sobre HTTPS; a validade é curta; nada sensível vai no payload (senha, documento, dados de cartão); e, quando a informação precisa estar atualizada, o servidor confere no banco em vez de confiar no token.

Um exemplo mostra a força da integridade. Se alguém decodifica o payload, troca `"ROLE_CLIENT"` por `"ROLE_ADMIN"` e codifica de novo, a assinatura calculada pelo servidor sobre o novo conteúdo não bate com a assinatura enviada. O token é recusado com 401.

CUIDADO COM O CAMPO ALG

O header diz qual algoritmo foi usado, mas é o *servidor* que deve decidir quais aceita. Um validador que obedece cegamente ao `alg` do token abre brecha para ataques em que o atacante declara um algoritmo mais fraco ou nenhum. Configure o validador para aceitar só o algoritmo esperado, como o ServiceHub faz com `macAlgorithm(MacAlgorithm.HS256)`.

5. HS256 e RS256

↑ conteúdo

A assinatura pode ser feita de duas formas. No **HS256**, a assinatura é um **HMAC** (*Hash-based Message Authentication Code*): o algoritmo mistura um segredo com o conteúdo e aplica SHA-256. Quem assina e quem confere usam o mesmo segredo. No **RS256**, a assinatura é assimétrica, com RSA: um par de chaves em que a privada assina e a pública confere.

Critério	HS256 (HMAC + SHA-256)	RS256 (RSA + SHA-256)
Chaves	Um segredo compartilhado	Par: chave privada e chave pública
Assina com	O segredo	A chave privada
Confere com	O mesmo segredo	A chave pública
Quem confere	Também consegue emitir tokens	Só confere; não consegue emitir
Quando usar	Um único serviço emite e valida	Muitos serviços validam o que um emissor assina

Exemplo geral: uma empresa com vinte microsserviços e um servidor de login central. Se usasse HS256, os vinte precisariam conhecer o segredo, e qualquer um deles vazando permitiria forjar tokens para todos. Com RS256, o servidor de login guarda a chave privada e publica a pública; os vinte conferem, mas nenhum consegue emitir.

No ServiceHub, a própria API emite e valida o token. Não há terceiro envolvido, então HS256 é suficiente e mais simples. O segredo vem da variável `JWT_SECRET` e precisa ter pelo menos 32 bytes. O número não é arbitrário: o SHA-256 produz 256 bits (32 bytes), e o padrão de algoritmos JWA pede para o HS256 uma chave de pelo menos esse tamanho. Um segredo menor que a saída do hash enfraquece a assinatura.

PARA LEMBRAR

Se a API um dia passar a aceitar tokens de um provedor externo (por exemplo, "entrar com o Google"), quem assina passa a ser o provedor, com chave assimétrica. A API troca o decoder para validar com a chave pública do provedor; o resto da cadeia de segurança continua igual.

6. Claims registradas e privadas

↑ conteúdo

A RFC 7519 reserva alguns nomes de claim com significado fixo, para que bibliotecas diferentes se entendam. Todas são opcionais; o padrão só garante que, se aparecerem, querem dizer a mesma coisa.

Claim	Nome	Significado
`iss`	Issuer	Quem emitiu o token
`sub`	Subject	De quem o token fala (o usuário)
`aud`	Audience	Para quem o token se destina
`exp`	Expiration time	Depois deste instante, o token não vale
`nbf`	Not before	Antes deste instante, o token ainda não vale
`iat`	Issued at	Quando foi emitido
`jti`	JWT ID	Identificador único do token

Os tempos são números: segundos desde 1º de janeiro de 1970 em UTC (o "tempo Unix"). Além das registradas existem as **claims privadas**, combinadas entre quem emite e quem consome. Use nomes curtos, porque o token viaja em toda requisição.

No ServiceHub, o payload tem quatro claims registradas e duas privadas. Os valores numéricos abaixo são ilustrativos:

```
{
  "iss":   "servicehub-api",
  "sub":   "ana@exemplo.com",
  "uid":   42,
  "roles": ["ROLE_PROVIDER"],
  "iat":   1792764000,
  "exp":   1792850400
}
```

- `sub` é o e-mail; `uid` é o id do usuário no banco. O servidor busca o usuário pelo `uid`, porque o e-mail pode mudar e o id não.
- `roles` é o papel no momento da emissão. Ele serve ao cliente (um front-end pode esconder botões), mas o servidor não confia nele: o papel válido é o que está no banco agora.
- `exp` menos `iat` dá 86 400 segundos, as 24 horas padrão da variável `JWT_EXPIRATION_SECONDS`.

7. Expiração, revogação e refresh token

↑ conteúdo

Expiração

O claim `exp` limita o estrago de um token roubado: passado o prazo, ele não vale mais. Escolher o prazo é um equilíbrio. Curto demais obriga o usuário a fazer login o tempo todo; longo demais amplia a janela em que um atacante usa o token capturado.

Revogação

Um token sem estado não pode ser "apagado", porque o servidor não guarda nada sobre ele. Para invalidá-lo antes do `exp`, o servidor precisa consultar alguma coisa a cada requisição: uma lista de tokens bloqueados ou o próprio usuário no banco. Isso é, de propósito, um pouco de estado de volta, aplicado só onde ele compensa.

Refresh token

Muitos sistemas usam dois tokens. O **token de acesso** é curto (minutos) e vai em cada chamada. O **refresh token** é longo, fica registrado no servidor e só serve para pedir um novo token de acesso. Como o refresh é consultado no servidor, ele pode ser revogado; como o de acesso é curto, ele expira logo. O par reúne a leveza do token com a capacidade de revogar da sessão.

O custo do refresh são mais peças: endpoint de renovação, armazenamento e rotação do refresh, logout que o revoga. **No ServiceHub** há só token de acesso, de 24 horas, sem refresh. Para compensar, o servidor confere no banco a cada chamada se o usuário segue ativo: desativou, o próximo pedido já é 401 (RN11). Conferir o usuário no banco custa uma consulta por requisição autenticada. O projeto aceita esse custo e o registra no ADR-0003: em troca, desativar alguém ou trocar seu papel vale na hora, sem esperar o token vencer.

Refresh token é conceito desta semana, não entrega: o contrato não pede renovação de token.

8. OAuth 2.0 e resource server

↑ conteúdo

O **OAuth 2.0** é um padrão de *delegação de acesso*: ele permite que uma pessoa autorize um aplicativo a acessar dados dela em outro sistema, sem entregar a senha ao aplicativo. O exemplo do dia a dia é o botão "entrar com o Google" num site qualquer. O padrão define quatro papéis:

Papel	Quem é	No "entrar com o Google"
Dono do recurso	A pessoa cujos dados estão em jogo e que autoriza o acesso.	Você
Cliente	O aplicativo que quer acessar em nome da pessoa.	O site em que você entra
Servidor de autorização	Autentica a pessoa e emite o token de acesso.	O login do Google
Resource server	A API que guarda os dados e aceita o token como prova.	A API do Google que devolve seu perfil

O OAuth 2.0 trata de autorização delegada. O login federado propriamente dito, em que o site quer saber *quem* você é, é o **OpenID Connect**, construído por cima do OAuth 2.0. Não vamos implementar nenhum dos dois; o que interessa é o papel de **resource server**, porque é ele que o Spring Security oferece pronto.

No ServiceHub, os papéis se simplificam: a própria API emite o token no login (faz as vezes de servidor de autorização) e o valida em cada chamada (é o resource server). Daí o nome da dependência, `spring-boot-starter-oauth2-resource-server`: ela dá à API a capacidade de validar tokens Bearer.

O que o resource server faz em cada requisição



As quatro etapas da validação e a peça do Spring (ou do projeto) responsável por cada uma.

As três primeiras etapas respondem "sei quem você é?"; qualquer falha nelas é 401 (sem token, token adulterado, expirado, usuário inexistente ou inativo). A quarta responde "você pode isto?"; falha nela é 403.

9. Papel, dono e menor privilégio

↑ conteúdo

Autorização por papel (RBAC)

RBAC (*Role-Based Access Control*, controle de acesso baseado em papéis) agrupa permissões em papéis e atribui papéis a pessoas. A regra fica do tipo "só ADMIN lista usuários". A decisão depende só do usuário e pode ser tomada antes de carregar qualquer dado.

Autorização por dono do recurso

A segunda pergunta é sobre o objeto: "este serviço é seu?". A decisão depende do usuário *e* do objeto, então só pode ser tomada depois de carregar a entidade do banco. Quando essa checagem falta, a falha tem nome: **BOLA** (*Broken Object Level Authorization*, autorização quebrada no nível do objeto), o item API1 do OWASP API Security Top 10.

Critério	Por papel (RBAC)	Por dono do recurso
Pergunta	Este tipo de usuário pode esta operação?	Este objeto é deste usuário?
Depende de	Só do usuário	Do usuário e do objeto no banco
Onde decide	Antes de carregar dados	Depois de carregar a entidade
Se faltar	Usuário comum executa função de admin (API5 do OWASP, autorização quebrada no nível da função)	BOLA: ver ou alterar o que é de outro (API1)
ServiceHub	`@PreAuthorize("hasRole('ADMIN')")`	Só o prestador dono (ou ADMIN) altera o serviço

EXEMPLO FORA DO SERVICEHUB

Num hospital, ser médico (papel) permite abrir prontuários. Ser o médico daquele paciente (dono da relação) permite alterar aquele prontuário. Um sistema que só confere o papel deixa qualquer médico alterar o prontuário de qualquer paciente.

POR QUE BOLA ESCAPA

Com um usuário só, o código sem checagem de dono funciona perfeitamente: você cria um serviço e consegue editá-lo. A falha só aparece com **dois** usuários, quando Ana troca o id na URL e edita o serviço de Bruno. Por isso a prova desta semana sempre usa dois usuários do mesmo papel.

Os papéis do ServiceHub

Papel	O que faz
CLIENT	Busca serviços, abre ordens, cancela as suas e avalia o que foi concluído.
PROVIDER	Cadastra os próprios serviços e conduz as ordens recebidas. Também pode contratar serviços de outros.
ADMIN	Gerencia usuários e papéis e modera serviços. Não contrata serviços. O primeiro ADMIN nasce das variáveis `SERVICEHUB_ADMIN_*` (ADR-0007); o cadastro público aceita só CLIENT ou PROVIDER (RN03).

Se o corpo do cadastro aceitasse qualquer papel, qualquer pessoa se cadastraria como ADMIN. Esse ataque se chama **mass assignment**: o cliente preenche um campo que não deveria controlar. A defesa é o DTO de entrada aceitar só os campos permitidos e validar o valor.

Princípio do menor privilégio

Cada usuário, serviço ou componente recebe só o acesso de que precisa, e nada além. Exemplo geral: o estagiário recebe a chave da sala dele, não a chave mestra do prédio; se perder a chave, o estrago fica pequeno.

No ServiceHub o princípio aparece em vários lugares: o cadastro público não cria ADMIN; ADMIN não contrata; o Actuator completo é só para ADMIN; e, principalmente, a cadeia de filtros termina com `anyRequest().authenticated()`. Essa última linha é o menor privilégio aplicado à configuração: *negar por padrão e liberar por exceção*. Uma rota nova nasce protegida; para ser pública, alguém precisa liberá-la de propósito.

10. 401 ou 403

↑ conteúdo

A RFC 9110, que define a semântica do HTTP, separa os dois status pela pergunta que falhou.

Status	Significa	O cliente deve
401 Unauthorized	Não autenticado: falhou "sei quem você é?". Vem com o header `WWW-Authenticate`, que diz como se autenticar (`Bearer`).	Fazer login e repetir com um token válido.
403 Forbidden	Autenticado, mas sem permissão: falhou "você pode isto?".	Não repetir: outro token do mesmo usuário não muda nada.

CONFUSÃO COMUM

O nome "Unauthorized" engana: o 401 é sobre *autenticação*. Os dois erros típicos em código gerado por IA são devolver 403 para quem não mandou token e devolver 401 para quem tem token válido mas não é dono do objeto.

Algumas APIs respondem 404 em vez de 403 a quem não é dono, para não revelar que o objeto existe. É uma escolha válida desde que esteja no contrato. No ServiceHub, falta de posse é 403, e rota inexistente continua 404 mesmo sem token.

Os dois erros seguem o mesmo formato ProblemDetail (RFC 9457) do resto da API, com `type` próprio:

```
HTTP/1.1 401
WWW-Authenticate: Bearer
Content-Type: application/problem+json

{
  "type": "urn:servicehub:problem:unauthorized",
  "title": "Não autenticado",
  "status": 401,
  "detail": "Envie um token válido no header Authorization: Bearer <token>.",
  "instance": "/api/v1/auth/me"
}
```

O 403 tem o mesmo formato, com `"type": "urn:servicehub:problem:forbidden"`, `"title": "Acesso negado"` e `"detail": "Você não tem permissão para esta operação."`. O corpo acima está resumido: a resposta real também traz o campo `timestamp`.

11. Aplicação no ServiceHub

↑ conteúdo

Muitos tutoriais implementam JWT com a biblioteca `jjwt` e um filtro escrito à mão. O ServiceHub usa o suporte nativo do Spring Security 7, decisão registrada no ADR-0003. São uma dependência, três beans e um converter. Lembre o detalhe do Spring Boot 4: é o *starter* `spring-boot-starter-oauth2-resource-server` que liga a auto-configuração; ele traz junto a biblioteca Nimbus JOSE, que faz a parte criptográfica.

A cadeia de filtros

Trecho de `security/SecurityConfig.java`, resumido (sem CORS, Actuator e documentação):

```
http.csrf(csrf -> csrf.disable())
    .sessionManagement(s -> s.sessionCreationPolicy(STATELESS))
    .authorizeHttpRequests(auth -> auth
        .requestMatchers(HttpMethod.POST,
                "/api/v1/auth/register", "/api/v1/auth/login").permitAll()
        .requestMatchers(HttpMethod.GET, "/api/v1/services", "/api/v1/services/**",
                "/api/v1/reviews/**", "/api/v1/info").permitAll()
        .requestMatchers("/actuator/**").hasRole("ADMIN")
        .anyRequest().authenticated())
    .oauth2ResourceServer(oauth -> oauth
        .jwt(jwt -> jwt.jwtAuthenticationConverter(converter))
        .authenticationEntryPoint(problems.entryPoint())
        .accessDeniedHandler(problems.accessDeniedHandler()));
```

`STATELESS` elimina a sessão HTTP; com o token no header, CSRF pode ser desligado. `permitAll` libera login, cadastro e leitura do catálogo; todo o resto exige token. `oauth2ResourceServer` liga o filtro do Spring que lê o Bearer, valida com o `JwtDecoder` e chama o nosso converter. O `entryPoint` e o `accessDeniedHandler` vêm do `ProblemResponseWriter`, para que 401 e 403 gerados nos filtros (antes do controller) saiam no mesmo ProblemDetail do `GlobalExceptionHandler`. A classe também tem `@EnableMethodSecurity`, sem a qual `@PreAuthorize` é ignorado em silêncio.

Chave, encoder e decoder

```
@Bean SecretKey jwtSigningKey(JwtProperties properties) {
    return new SecretKeySpec(properties.secret().getBytes(UTF_8), "HmacSHA256");
}
@Bean JwtEncoder jwtEncoder(SecretKey jwtSigningKey) {
    return new NimbusJwtEncoder(new ImmutableSecret<>(jwtSigningKey));
}
@Bean JwtDecoder jwtDecoder(SecretKey jwtSigningKey) {
    return NimbusJwtDecoder.withSecretKey(jwtSigningKey)
            .macAlgorithm(MacAlgorithm.HS256).build();
}
```

O decoder valida assinatura e `exp` sozinho. É ele que substitui o método `isTokenValid` que os tutoriais escrevem à mão. O segredo vem de `JwtProperties`, um `record` anotado com `@ConfigurationProperties("servicehub.jwt")` cujo construtor compacto lança `IllegalStateException` se o segredo faltar ou tiver menos de `MIN_SECRET_BYTES = 32` bytes, e usa 86 400 segundos quando a expiração não é informada.

No `application.properties`, a linha é `servicehub.jwt.secret=${JWT_SECRET:}`: sem a variável a API não sobe, o que é melhor do que assinar tokens com chave fraca.

Login e emissão do token

O `AuthService.login` devolve a mesma resposta 401 ("E-mail ou senha inválidos.") para e-mail inexistente, senha errada e conta inativa, para não revelar quais e-mails existem. Quando o e-mail não existe, ele ainda compara a senha com um hash BCrypt fictício (`DUMMY_HASH`), para que o tempo de resposta também não denuncie. Com o usuário conferido, o `TokenService` monta as claims e assina:

```
Instant now = Instant.now();
JwtClaimsSet claims = JwtClaimsSet.builder()
        .issuer("servicehub-api")
        .subject(user.getEmail())
        .issuedAt(now)
        .expiresAt(now.plusSeconds(properties.expirationSeconds()))
        .claim(CLAIM_USER_ID, user.getId())                      // "uid"
        .claim(CLAIM_ROLES, List.of(user.getRole().authority()))  // "roles"
        .build();
JwsHeader header = JwsHeader.with(MacAlgorithm.HS256).build();
String token = encoder.encode(JwtEncoderParameters.from(header, claims)).getTokenValue();
return new TokenResponse(token, "Bearer", properties.expirationSeconds());
```

`Role.authority()` devolve `"ROLE_" + name()`, o formato que `hasRole('PROVIDER')` espera. A resposta do login fica `{"accessToken": "eyJ...", "tokenType": "Bearer", "expiresIn": 86400}`.

O converter: o usuário ainda está ativo?

```
public AbstractAuthenticationToken convert(Jwt jwt) {
    Object uid = jwt.getClaim(TokenService.CLAIM_USER_ID);
    if (!(uid instanceof Long || uid instanceof Integer)) {
        throw new DisabledException("Token sem identificação de usuário válida.");
    }
    User user = userRepository.findById(((Number) uid).longValue())
            .filter(User::isActive)
            .orElseThrow(() -> new DisabledException("Usuário inexistente ou desativado."));
    return new AuthUserAuthentication(
            new AuthUser(user.getId(), user.getEmail(), user.getRole()), jwt);
}
```

O `JwtUserConverter` recebe um JWT já validado e o transforma no usuário autenticado. Como lê o usuário do banco, ganha duas coisas: usuário desativado depois do login recebe 401 na hora (RN11), e as authorities vêm do papel atual do banco, não do claim `roles`. Se confiasse no claim, um ADMIN rebaixado continuaria ADMIN por até 24 horas.

@BEAN, NÃO @COMPONENT

O converter é registrado com `@Bean` no `SecurityConfig`. Com `@Component`, os testes de fatia `@WebMvcTest` (Semana 10), que carregam todo `Converter`, tentariam criá-lo sem repositório e quebrariam.

Regra de papel: @PreAuthorize no service

```
/** RN04: o prestador é sempre o usuário autenticado. */
@PreAuthorize("hasRole('PROVIDER')")
public ServiceOfferingResponseDTO createService(ServiceOfferingDTO dto) {
    User provider = userRepository.findById(currentUser.get().id()).orElseThrow(...);
    ...
    service.setProvider(provider);
    ...
}
```

A anotação fica no *service*, para valer em qualquer caminho que chame o método, e o dono vem do token via `CurrentUser`, nunca do corpo. Outros usos: `hasAnyRole('CLIENT', 'PROVIDER')` para abrir ordem e `hasRole('ADMIN')` na gestão de usuários.

Regra de dono: checagem explícita depois de carregar

```
private boolean canManage(ServiceOffering service) {
    return currentUser.find()
            .map(u -> u.isAdmin() || u.id().equals(service.getProvider().getId()))
            .orElse(false);
}
private void requireOwnerOrAdmin(ServiceOffering service) {
    if (!canManage(service)) {
        throw new AccessDeniedException(
                "Apenas o prestador dono ou um ADMIN pode alterar este serviço.");
    }
}
```

`@PreAuthorize` não sabe de quem é o objeto; a checagem de dono vem depois de buscar a entidade e lança `AccessDeniedException` (403). O mesmo padrão aparece em `UserService.requireSelfOrAdmin` (RN13), nas ordens e nas avaliações (RN09).

Do tutorial ao nativo

Mesmos conceitos, menos código seu; o `@Value("${jwt.secret}")` dos tutoriais vira o `JwtProperties`. Quem preferir `jjwt` pode usá-lo na API atual, justificando num ADR.

Com jjwt e filtro manual	Com o Spring Security nativo
`JwtService.generateToken`	`TokenService` com `JwtEncoder` (`NimbusJwtEncoder`, HS256)
`JwtAuthFilter extends OncePerRequestFilter`	`BearerTokenAuthenticationFilter` do Spring, ligado por `.oauth2ResourceServer(jwt)`
`jwtService.isTokenValid`	`NimbusJwtDecoder` valida assinatura e `exp`
`loadUserByUsername` dentro do filtro	`JwtUserConverter` carrega por `uid` e rejeita inativos

12. Construindo com IA

↑ conteúdo

Segurança é onde o código gerado por IA mais engana: passa no teste com um usuário e deixa a porta aberta para o segundo.

Movimento	O que você faz nesta semana
Especificar	Tabela "papel × rota × dono" tirada do Mapa, com os casos 401 e 403 esperados.
Gerar	Plano primeiro; starter do resource server, sem jjwt nem filtro manual; código por partes.
Revisar	Segredo fora do código, papel do banco, dono checado no service, erros em ProblemDetail.
Provar	Dois usuários do mesmo papel; usuário desativado repetindo a chamada com o token antigo.

Prompt fraco e prompt bom

Fraco

```
adiciona login com JWT na API
```

Volta com jjwt antigo, filtro manual, segredo fixo e nenhuma regra de dono.

Bom

Fixa versões, starter, claims e segredo, proíbe jjwt, cita as RN e pede plano. É o prompt abaixo.

O prompt desta semana

A entrega é o login com JWT, as rotas protegidas por papel e por dono, e a prova com dois usuários. O prompt parte da API da Semana 08 (cadastro com BCrypt e validação já prontos):

```
Contexto: API ServiceHub em Java 21, Spring Boot 4.1, Spring Security 7,
PostgreSQL 16 + Flyway. Já existem POST /api/v1/auth/register (só CLIENT
ou PROVIDER), BCrypt, SecurityConfig STATELESS e ProblemDetail global.

Objetivo: POST /api/v1/auth/login (público) devolvendo {accessToken,
tokenType: "Bearer", expiresIn}, GET /api/v1/auth/me e as rotas
protegidas conforme a tabela de endpoints do Mapa (colada no fim).

Restrições:
- spring-boot-starter-oauth2-resource-server com NimbusJwtEncoder e
  NimbusJwtDecoder.withSecretKey(...).macAlgorithm(HS256). NÃO use
  jjwt, OncePerRequestFilter manual nem WebSecurityConfigurerAdapter.
- servicehub.jwt.secret=${JWT_SECRET:} lido por um record
  @ConfigurationProperties que impede a subida sem segredo ou com
  menos de 32 bytes. Nenhum segredo no código.
- Claims iss, sub (e-mail), iat, exp (24 h), uid e roles. Sem refresh.
```

```
  STATELESS; anyRequest().authenticated() no fim da cadeia.
- Converter<Jwt, AbstractAuthenticationToken> como @Bean (não
  @Component): busca o usuário pelo claim uid, rejeita inexistente ou
  inativo (RN11) e usa o papel do BANCO, não o claim roles.
- Login: mesma resposta 401 para e-mail inexistente, senha errada e
  conta inativa.
- Papel: @PreAuthorize nos métodos do SERVICE (@EnableMethodSecurity).
  Dono (RN05 serviço, RN13 perfil): AccessDeniedException explícita no
  service, depois de carregar a entidade. Prestador vem do token.
- 401 (WWW-Authenticate: Bearer) e 403 em ProblemDetail, nos filtros
  e no GlobalExceptionHandler.

Antes do código, mostre o plano: arquivos a criar ou alterar e a
tabela papel x rota x dono que você entendeu. Espere minha aprovação.

Critério de aceite: testes de integração com
(a) rota protegida sem token: 401 em ProblemDetail;
(b) token com um caractere alterado: 401;
(c) CLIENT em POST /api/v1/services: 403;
(d) PROVIDER Ana em PUT no serviço do PROVIDER Bruno: 403; Bruno: 200;
(e) usuário desativado pelo ADMIN, com o token que já tinha: 401.

Endpoints do Mapa: <cole aqui>
```

O que a IA costuma errar nesta etapa

- Usar `WebSecurityConfigurerAdapter` (removido) ou `jjwt` com `setClaims` e `signWith(key, SignatureAlgorithm)`, APIs antigas e depreciadas.
- Segredo fixo ou curto; authorities lidas do claim `roles` em vez do banco.
- Só `@PreAuthorize`, sem checagem de dono; `providerId` lido do corpo.
- Converter com `@Component`; 401 e 403 com corpo vazio ou HTML, ou trocados.

Checklist de revisão

☐ O `pom.xml` tem `spring-boot-starter-oauth2-resource-server` e não tem `jjwt`?

☐ A API se recusa a subir sem `JWT_SECRET` ou com segredo de menos de 32 bytes?

☐ O converter busca por `uid`, rejeita inativos e usa o papel do banco? É `@Bean`?

☐ Cada alteração de serviço e de perfil tem checagem de dono no service?

Prova

Testes do critério de aceite verdes. À mão, com o serviço 1 do Bruno e o token da Ana, a resposta tem de ser 403 em `application/problem+json`; depois de o ADMIN desativar Ana, `GET /api/v1/auth/me` com o token antigo dela tem de dar 401.

```
curl -i -X PUT localhost:8090/api/v1/services/1 -H "Authorization: Bearer $TOKEN_ANA" \
  -H "Content-Type: application/json" -d '{"title":"Faxina","price":120,"category":"LIMPEZA"}'
```

13. Glossário

↑ conteúdo

Autenticação	Conferir quem está chamando, a partir de uma prova de identidade.
Autorização	Decidir se quem já foi identificado pode executar a operação sobre aquele objeto.
Sessão	Registro guardado no servidor sobre um cliente, identificado por um id enviado em cookie.
JWT	JSON Web Token: texto compacto com header, payload e assinatura (RFC 7519).
Claim	Afirmação dentro do payload, como `sub` ou `exp`.
Base64URL	Codificação reversível de bytes em texto seguro para URL; não é criptografia.
HMAC	Código de autenticação de mensagem que combina um segredo com um hash; base do HS256.
HS256 / RS256	Assinatura com segredo compartilhado / com par de chaves privada e pública.
Bearer	Esquema do header `Authorization`: quem porta o token tem o acesso.
Refresh token	Token longo, guardado no servidor, usado só para obter novo token de acesso.
OAuth 2.0	Padrão de delegação de acesso com dono do recurso, cliente, servidor de autorização e resource server.
Resource server	API que aceita um token de acesso como prova e protege os dados.
RBAC	Controle de acesso baseado em papéis.
BOLA	Falha em que um usuário acessa objeto de outro porque falta a checagem de dono (OWASP API1).
Menor privilégio	Dar a cada parte só o acesso de que ela precisa; negar por padrão.
Mass assignment	Ataque em que o cliente preenche campos que não deveria controlar, como o papel.

14. Exercícios

↑ conteúdo

1. Explique a diferença entre autenticação e autorização e dê, para cada uma, um exemplo de falha no ServiceHub com o status HTTP correspondente.
2. Uma API roda em quatro instâncias atrás de um balanceador. Compare o que é preciso fazer para manter o usuário logado com sessão no servidor e com JWT.
3. Decodifique mentalmente (ou no console) o trecho `eyJzdWIiOiJhbmEifQ`. O que isso prova sobre o sigilo do payload? Cite duas coisas que nunca devem ir no payload.
4. Um colega diz: "como o token é assinado, posso guardar nele o papel e nunca mais consultar o banco". Aponte o problema usando o que a assinatura não garante e diga como o ServiceHub resolve.

5. Uma empresa tem um servidor de login e quinze APIs que precisam validar os tokens. HS256 ou RS256? Justifique pelo risco de vazamento.
6. Por que `JwtProperties` exige pelo menos 32 bytes? O que acontece se `JWT_SECRET` não estiver definida, e por que isso é desejável?
7. Analise o código: `@PreAuthorize("hasRole('PROVIDER')") public void update(Long id, ServiceOfferingDTO dto) { var s = repo.findById(id).orElseThrow(); apply(s, dto); }`. Qual regra do Mapa ele viola, qual o nome da falha e como corrigir?
8. Classifique cada caso em 401, 403 ou 404: (a) GET `/api/v1/auth/me` sem header; (b) CLIENT faz GET `/api/v1/users`; (c) token com `exp` vencido; (d) PROVIDER apaga serviço de outro; (e) GET `/api/v1/rota-que-nao-existe` sem token; (f) usuário desativado usa token emitido ontem.
9. Por que o `JwtUserConverter` é registrado como `@Bean` e não como `@Component`? Que tipo de teste quebraria?
10. Explique o que é refresh token e que problema ele resolve. Por que o ServiceHub consegue ficar sem ele sem perder a capacidade de bloquear um usuário?
11. Escreva um prompt para pedir à IA apenas os testes de integração de BOLA da rota `PUT /api/v1/users/{id}` (RN13). Indique os dois usuários envolvidos, os status esperados e o que você conferiria no código gerado.

15. Para estudar

↑ conteúdo

Leituras disponíveis na Minha Biblioteca, pelo acesso institucional:

- RODRIGUES, T. N. et al. *Integração de aplicações*. Grupo A, 2020. Capítulo "Segurança em webservices".
- PRESSMAN, R. S.; MAXIM, B. R. *Engenharia de software*. 9. ed. Grupo A, 2021. Capítulo 18 (engenharia de segurança), em revisão.

Referências abertas:

- RFC 7519, *JSON Web Token (JWT)*: seções 3 (estrutura) e 4.1 (claims registradas). `rfc-editor.org`. Leia com um token real do projeto decodificado ao lado.
- RFC 9110, *HTTP Semantics*: status 401 e 403.
- Documentação do Spring Security: *OAuth 2.0 Resource Server — JWT* e *Method Security* (`@PreAuthorize`). `docs.spring.io/spring-security`.
- OWASP API Security Top 10: API1 (Broken Object Level Authorization) e API5 (Broken Function Level Authorization). `owasp.org`.